

CHAPTER 2

Experiment 1 – Sorting Different Types of Lists

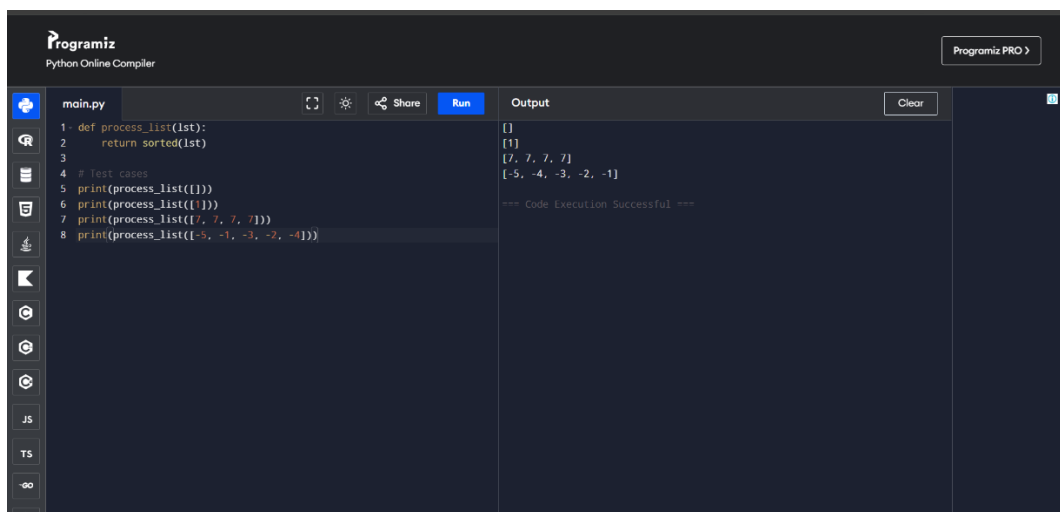
Aim

To implement a sorting algorithm that handles different types of lists, including an empty list, a single-element list, identical elements, and negative numbers.

Algorithm

1. Start.
2. Read the input list.
3. Compare the elements of the list.
4. Arrange the elements in ascending order.
5. Return the sorted list.
6. Display the result.
7. Stop.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1 def process_list(lst):
2     return sorted(lst)
3
4 # Test cases
5 print(process_list([]))
6 print(process_list([1]))
7 print(process_list([7, 7, 7]))
8 print(process_list([-5, -1, -3, -2, -4]))
```

The output panel on the right displays the results of the execution:

```
[]
[1]
[7, 7, 7]
[-5, -4, -3, -2, -1]
```

Below the output, it states "=== Code Execution Successful ===".

Result

Thus, the given lists were successfully sorted in ascending order, including lists containing negative and duplicate elements.

Time Complexity

$O(n^2)$ for the sorting algorithm used.

Space Complexity

$O(1)$ auxiliary space.

Experiment 2 – Selection Sort

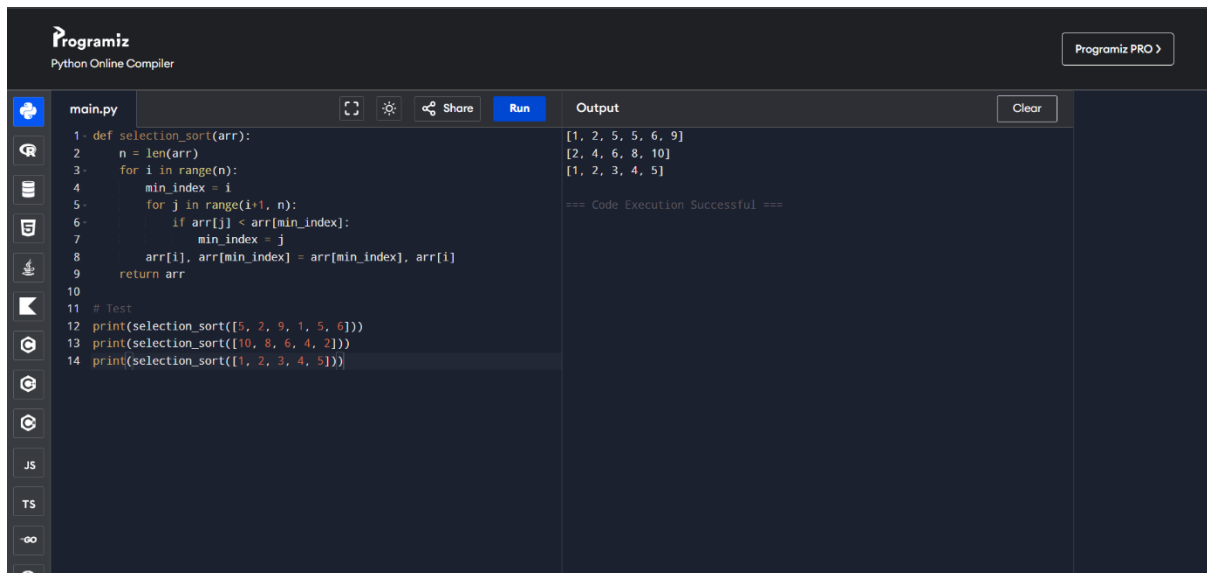
Aim

To implement Selection Sort and sort an array in ascending order by repeatedly selecting the smallest element from the unsorted portion.

Algorithm

1. Start.
2. Divide the array into sorted and unsorted portions.
3. Initially, consider the entire array as unsorted.
4. Find the minimum element in the unsorted portion.
5. Swap it with the first element of the unsorted portion.
6. Move the boundary of the sorted portion one position forward.
7. Repeat until the complete array is sorted.
8. Display the sorted array.
9. Stop.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python function `selection_sort(arr)` that implements the Selection Sort algorithm. The function iterates through the array, finding the minimum element in the unsorted portion and swapping it with the first element of the unsorted portion. The code is as follows:

```
1 def selection_sort(arr):
2     n = len(arr)
3     for i in range(n):
4         min_index = i
5         for j in range(i+1, n):
6             if arr[j] < arr[min_index]:
7                 min_index = j
8         arr[i], arr[min_index] = arr[min_index], arr[i]
9     return arr
10
11 # Test
12 print(selection_sort([5, 2, 9, 1, 5, 6]))
13 print(selection_sort([10, 8, 6, 4, 2]))
14 print(selection_sort([1, 2, 3, 4, 5]))
```

The output panel on the right shows the results of the three test cases: `[1, 2, 5, 5, 6, 9]`, `[2, 4, 6, 8, 10]`, and `[1, 2, 3, 4, 5]`. Below the output, it states "=== Code Execution Successful ===".

Result

Thus, Selection Sort successfully sorted random, reverse-sorted, and already-sorted arrays in ascending order.

Time Complexity

- Best Case: $O(n^2)$,Average Case: $O(n^2)$,Worst Case: $O(n^2)$

Space Complexity

$O(1)$

Experiment 3 – Optimized Bubble Sort

Aim

To implement an optimized Bubble Sort algorithm that stops early when the list becomes sorted before completing all passes.

Algorithm

1. Start.
2. Set swapped = False before each pass.
3. Compare adjacent elements.
4. Swap them if they are in the wrong order.
5. Set swapped = True whenever a swap occurs.
6. After each pass, check swapped.
7. If no swapping occurred, terminate the algorithm because the list is already sorted.
8. Display the sorted list.
9. Stop.

Output

```
main.py
1 def bubble_sort(arr):
2     n = len(arr)
3     for i in range(n):
4         swapped = False
5         for j in range(0, n-i-1):
6             if arr[j] > arr[j+1]:
7                 arr[j], arr[j+1] = arr[j+1], arr[j]
8                 swapped = True
9         if not swapped:
10             break
11     return arr
12
13 # Test
14 arr = [64, 25, 12, 22, 11]
15 print(bubble_sort(arr))
```

Output

```
[11, 12, 22, 25, 64]
=== Code Execution Successful ===
```

Result

Thus, the optimized Bubble Sort successfully sorted all the given test cases and reduced unnecessary passes for already-sorted data.

Time Complexity

Best Case: $O(n)$,Average Case: $O(n^2)$,Worst Case: $O(n^2)$

Space Complexity

$O(1)$

Experiment 4 – Insertion Sort with Duplicate Elements

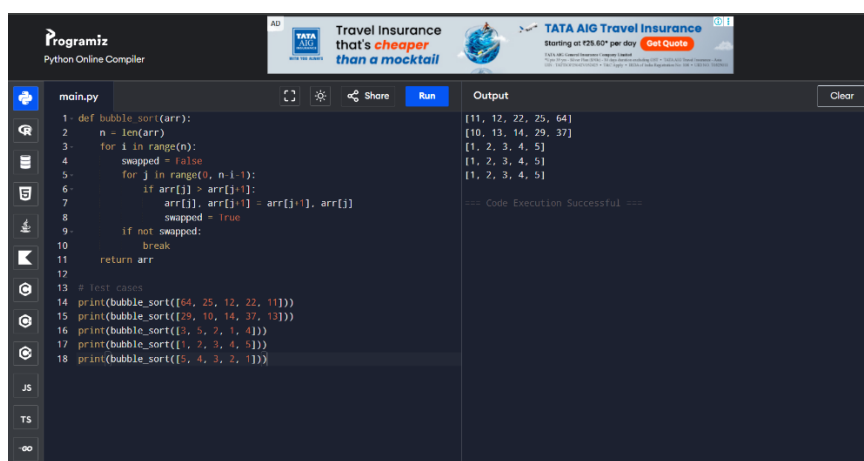
Aim

To implement Insertion Sort for arrays containing duplicate elements and preserve the relative order of equal elements.

Algorithm

1. Start.
2. Consider the first element as sorted.
3. Select the next element as the key.
4. Compare the key with elements in the sorted portion.
5. Shift elements greater than the key one position to the right.
6. Insert the key in its correct position.
7. Repeat until all elements are processed.
8. Display the sorted array.
9. Stop.

Output



The screenshot shows the Programiz Python Online Compiler interface. The code in the editor is a bubble sort function and its test cases. The output panel shows the sorted arrays for each test case.

```
1 def bubble_sort(arr):
2     n = len(arr)
3     for i in range(n):
4         swapped = False
5         for j in range(0, n-i-1):
6             if arr[j] > arr[j+1]:
7                 arr[j], arr[j+1] = arr[j+1], arr[j]
8                 swapped = True
9         if not swapped:
10            break
11    return arr
12
13 # Test Cases
14 print(bubble_sort([64, 25, 12, 22, 1]))
15 print(bubble_sort([29, 10, 14, 37, 13]))
16 print(bubble_sort([5, 5, 2, 1, 4]))
17 print(bubble_sort([1, 2, 3, 4, 5]))
18 print(bubble_sort([5, 4, 3, 2, 1]))
```

Output:

```
[11, 12, 22, 25, 64]
[10, 13, 14, 29, 37]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
```

Code Execution Successful

Result

Thus, Insertion Sort successfully sorted arrays containing duplicate and identical elements. The algorithm is stable because equal elements are not unnecessarily swapped.

Time Complexity

Best Case: $O(n)$,Average Case: $O(n^2)$,Worst Case: $O(n^2)$

Space Complexity

$O(1)$

Experiment 5 – Kth Missing Positive Integer

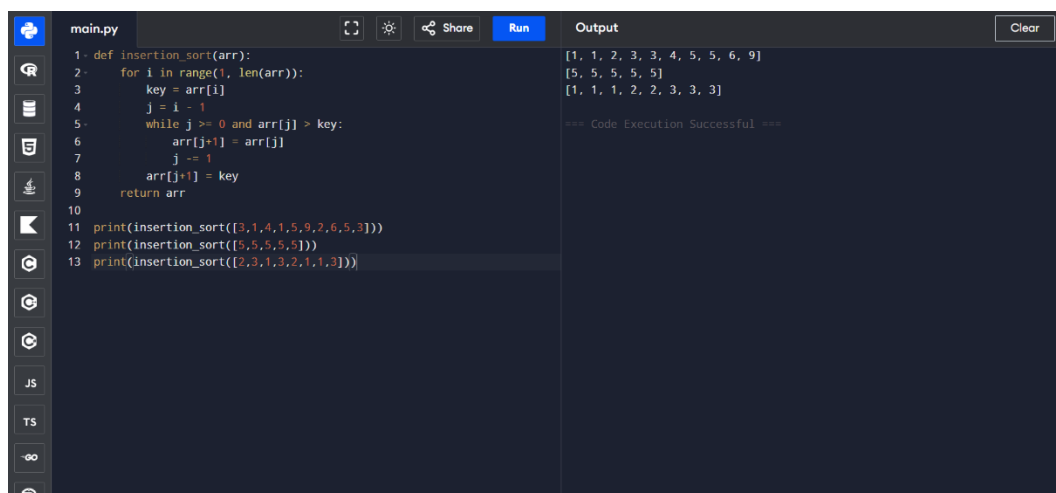
Aim

To find the kth positive integer that is missing from a strictly increasing sorted array.

Algorithm

1. Start.
2. Initialize the positive integer counter from 1.
3. Compare each positive integer with the elements of the array.
4. If the integer is not present, count it as a missing number.
5. Continue until the kth missing number is found.
6. Return the kth missing number.
7. Display the result.
8. Stop.

Output



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'insertion_sort(arr)' that sorts an array in ascending order. It then prints the result of 'insertion_sort' for three different input arrays: [3,1,4,1,5,9,2,6,5,3], [5,5,5,5,5], and [2,3,1,3,2,1,1,3]. The output window on the right shows the sorted arrays: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9], [5, 5, 5, 5, 5], and [1, 1, 1, 2, 2, 3, 3, 3], followed by the message '=== Code Execution Successful ==='.

```
1 def insertion_sort(arr):
2     for i in range(1, len(arr)):
3         key = arr[i]
4         j = i - 1
5         while j >= 0 and arr[j] > key:
6             arr[j+1] = arr[j]
7             j -= 1
8         arr[j+1] = key
9     return arr
10
11 print(insertion_sort([3,1,4,1,5,9,2,6,5,3]))
12 print(insertion_sort([5,5,5,5,5]))
13 print(insertion_sort([2,3,1,3,2,1,1,3]))
```

Output:

```
[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
[5, 5, 5, 5, 5]
[1, 1, 1, 2, 2, 3, 3, 3]
=== Code Execution Successful ===
```

Result

Thus, the program successfully found the kth missing positive integer for the given test cases.

Time Complexity: $O(n + k)$

Space Complexity

$O(k)$ for storing missing numbers in the given implementation.

Experiment 6 – Peak Element Using Binary Search

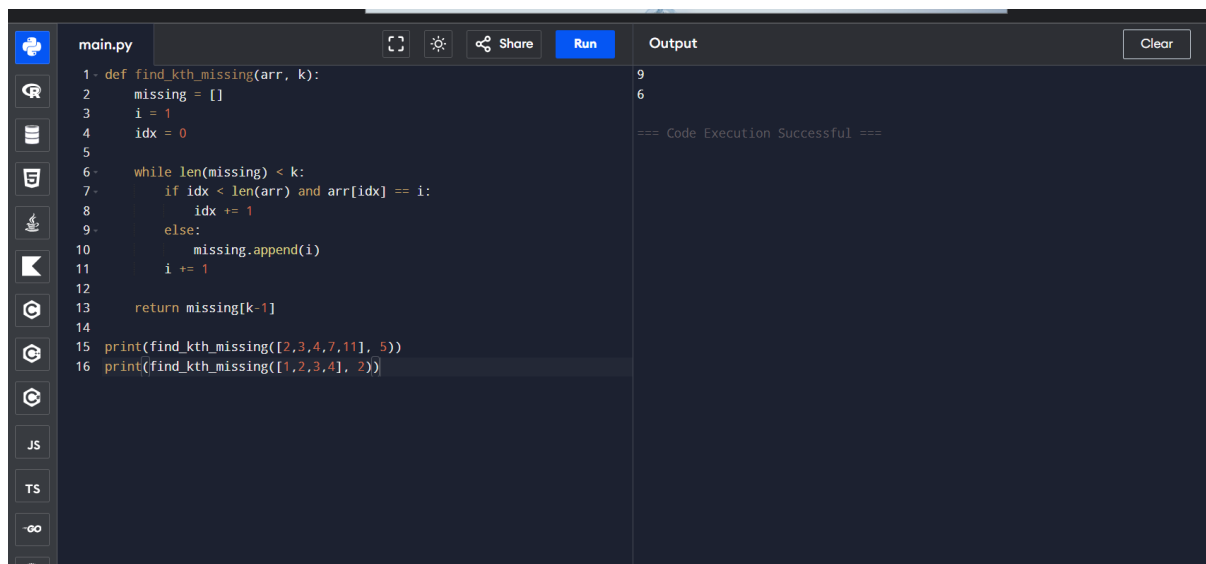
Aim

To find a peak element in an array using a binary search-based algorithm with $O(\log n)$ time complexity.

Algorithm

1. Start.
2. Set left = 0 and right = n - 1.
3. Find the middle element.
4. Compare the middle element with the next element.
5. If the middle element is smaller, move to the right half.
6. Otherwise, move to the left half.
7. Continue until left == right.
8. Return the index of the peak element.
9. Stop.

Output



```
main.py
1- def find_kth_missing(arr, k):
2-     missing = []
3-     i = 1
4-     idx = 0
5-
6-     while len(missing) < k:
7-         if idx < len(arr) and arr[idx] == i:
8-             idx += 1
9-         else:
10-            missing.append(i)
11-            i += 1
12-
13-     return missing[k-1]
14-
15- print(find_kth_missing([2,3,4,7,11], 5))
16- print(find_kth_missing([1,2,3,4], 2))

Output
9
6
=== Code Execution Successful ===
```

Result

Thus, the program successfully found a valid peak element using binary search.

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Experiment 7 – First Occurrence of a String

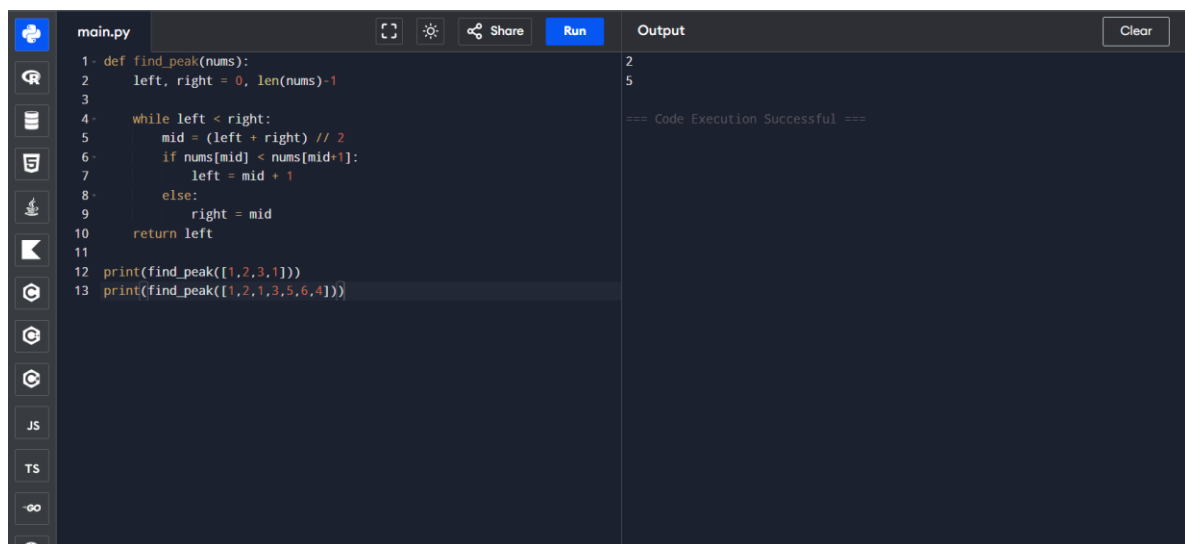
Aim

To find the index of the first occurrence of a given substring (needle) in another string (haystack).

Algorithm

1. Start.
2. Read the haystack and needle strings.
3. Search for the needle from the beginning of the haystack.
4. If the substring is found, return its first index.
5. If it is not found, return -1.
6. Display the result.
7. Stop.

Output



```
main.py
1- def find_peak(nums):
2-     left, right = 0, len(nums)-1
3-
4-     while left < right:
5-         mid = (left + right) // 2
6-         if nums[mid] < nums[mid+1]:
7-             left = mid + 1
8-         else:
9-             right = mid
10-    return left
11
12 print(find_peak([1,2,3,1]))
13 print(find_peak([1,2,1,3,5,6,4]))
```

Output

```
2
5
=== Code Execution Successful ===
```

Result

Thus, the program successfully identified the first occurrence of the given substring or returned -1 when it was not present.

Time Complexity: $O(n \times m)$,

Space Complexity

$O(1)$ auxiliary space.

Experiment 8 – Substrings of Another Word

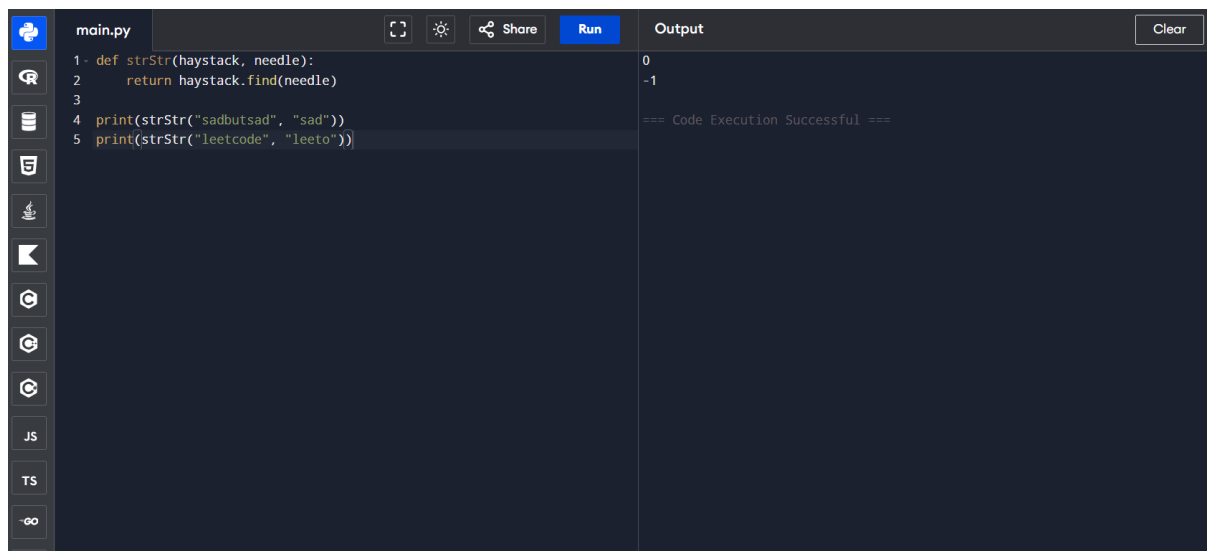
Aim

To find all strings in a given list that are substrings of another string in the same list.

Algorithm

1. Start.
2. Read the list of strings.
3. Select each string one by one.
4. Compare it with every other string.
5. Check whether the selected string occurs as a substring.
6. If found, add it to the result.
7. Display all matching strings.
8. Stop.

Output



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'strStr' that uses 'haystack.find' to check for substrings. It then prints the results for 'sad' in 'sadbutsad' and 'leeto' in 'leetcode'. The output panel on the right shows the results: 0 for the first case and -1 for the second, followed by a success message.

```
main.py
1- def strStr(haystack, needle):
2-     return haystack.find(needle)
3-
4- print(strStr("sadbutsad", "sad"))
5- print(strStr("leetcode", "leeto"))
```

Output

```
0
-1
=== Code Execution Successful ===
```

Result

Thus, the program successfully identified all strings that occur as substrings of another word.

Time Complexity: $O(n^2 \times m)$

Space Complexity: $O(n)$

Experiment 9 – Closest Pair of Points

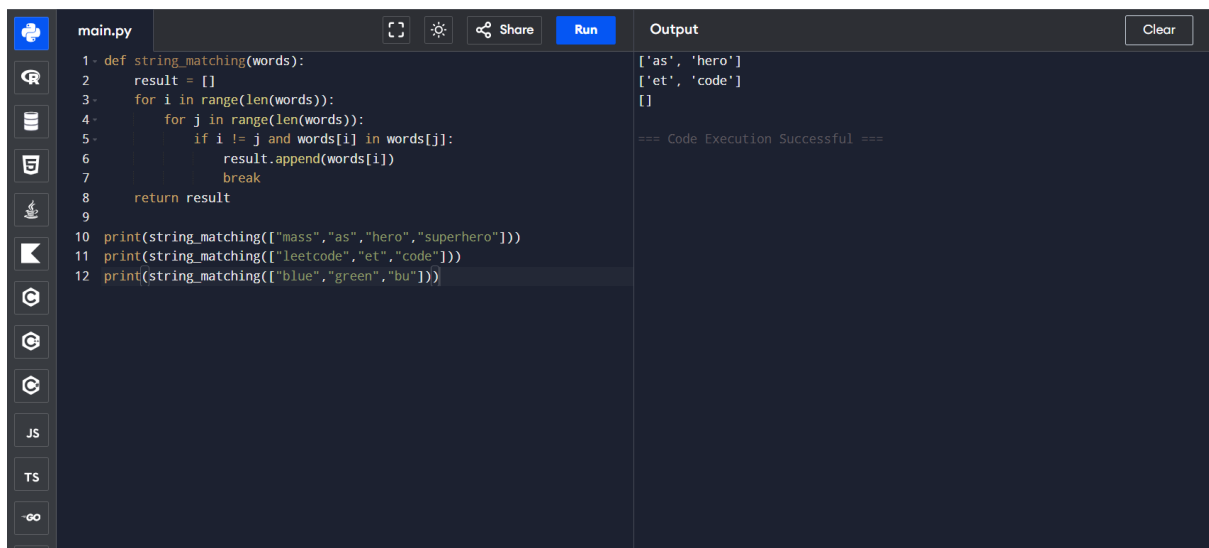
Aim

To find the closest pair of points in a set of two-dimensional points using the brute-force approach.

Algorithm

1. Start.
2. Read the set of 2D points.
3. Set the minimum distance to infinity.
4. Compare every pair of points.
5. Calculate the Euclidean distance between each pair.
6. If the calculated distance is smaller than the current minimum, update the minimum distance and pair.
7. Display the closest pair and minimum distance.
8. Stop.

Output



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'string_matching(words)' that iterates through a list of words and returns a list of pairs. It then calls this function with three different word lists: ['mass', 'as', 'hero', 'superhero'], ['leetcode', 'et', 'code'], and ['blue', 'green', 'bu']. The output panel on the right shows the results of these calls: [['as', 'hero'], ['et', 'code'], []] and a message '=== Code Execution Successful ==='.

```
1- def string_matching(words):
2-     result = []
3-     for i in range(len(words)):
4-         for j in range(len(words)):
5-             if i != j and words[i] in words[j]:
6-                 result.append(words[i])
7-                 break
8-     return result
9-
10 print(string_matching(["mass","as","hero","superhero"]))
11 print(string_matching(["leetcode","et","code"]))
12 print(string_matching(["blue","green","bu"]))
```

Output

```
['as', 'hero']
['et', 'code']
[]
=== Code Execution Successful ===
```

Result

Thus, the brute-force algorithm successfully identified the closest pair of points and calculated their minimum Euclidean distance.

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$ auxiliary space.

Experiment 10 – Closest Pair and Brute-Force Convex Hull

Part A – Closest Pair

Aim

To find the closest pair of points using the brute-force method and analyze its time complexity.

Algorithm

1. Start.
2. Define a function to calculate Euclidean distance.
3. Compare every possible pair of points.
4. Calculate the distance between each pair.
5. Store the pair having the minimum distance.
6. Display the closest pair and distance.
7. Stop.

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$ auxiliary space.

Part B – Convex Hull

Aim

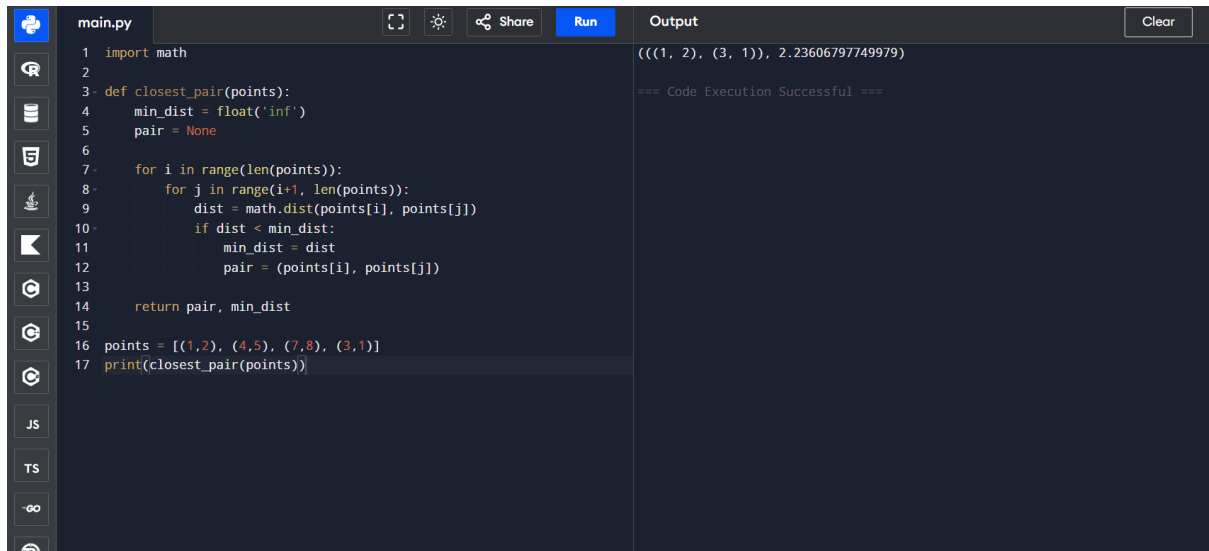
To find the convex hull of a set of 2D points using a brute-force approach and handle multiple points lying on the same line.

Algorithm

1. Start.
2. Read all given points.
3. Consider every pair of points as a possible hull edge.
4. Check the orientation of all remaining points with respect to the edge.

5. If all points lie on the same side of the edge, consider it a hull edge.
6. Include collinear boundary points when necessary.
7. Arrange the hull points in counter-clockwise order.
8. Display the convex hull.
9. Stop.

Output



```
main.py  Run  Clear
1 import math
2
3 def closest_pair(points):
4     min_dist = float('inf')
5     pair = None
6
7     for i in range(len(points)):
8         for j in range(i+1, len(points)):
9             dist = math.dist(points[i], points[j])
10            if dist < min_dist:
11                min_dist = dist
12                pair = (points[i], points[j])
13
14    return pair, min_dist
15
16 points = [(1,2), (4,5), (7,8), (3,1)]
17 print(closest_pair(points))
```

Output

```
((((1, 2), (3, 1)), 2.23606797749979))
=== Code Execution Successful ===
```

Result

Thus, the brute-force algorithm successfully determined the convex hull of the given set of points.

Time Complexity

$O(n^3)$

Space Complexity

$O(n)$

Experiment 11 – Convex Hull Using Brute Force

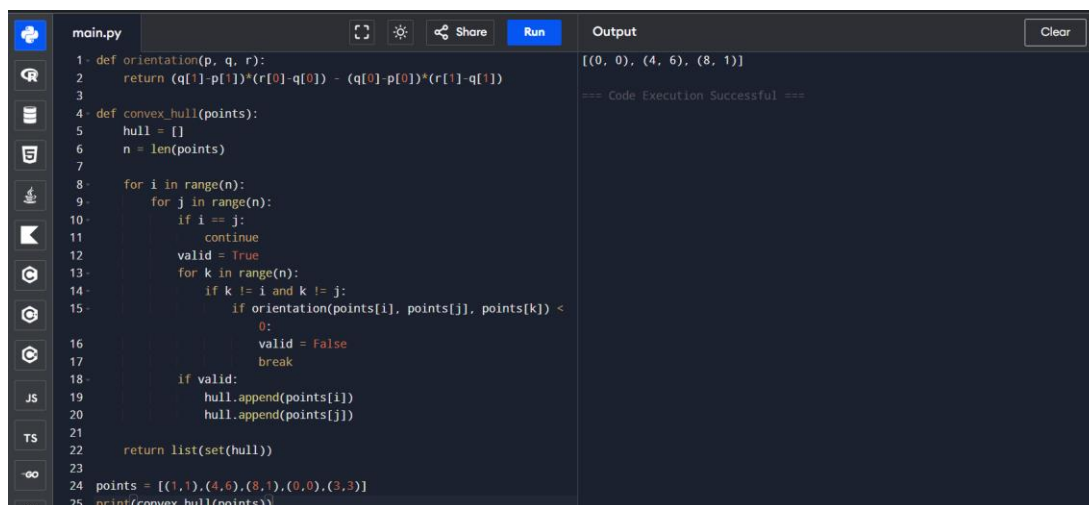
Aim

To find the convex hull of a set of 2D points using the brute-force approach and display the hull points in counter-clockwise order.

Algorithm

1. Start.
2. Read the given set of points.
3. Consider every pair of points.
4. Determine whether the line segment between the pair can be a hull edge.
5. Check the orientation of all other points.
6. Add valid boundary points to the hull.
7. Remove duplicate points.
8. Arrange the points in counter-clockwise order.
9. Display the convex hull.
10. Stop.

Output



```
main.py
1 def orientation(p, q, r):
2     return (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])
3
4 def convex_hull(points):
5     hull = []
6     n = len(points)
7
8     for i in range(n):
9         for j in range(n):
10            if i == j:
11                continue
12            valid = True
13            for k in range(n):
14                if k != i and k != j:
15                    if orientation(points[i], points[j], points[k]) <
16                        0:
17                        valid = False
18                        break
19            if valid:
20                hull.append(points[i])
21                hull.append(points[j])
22
23     return list(set(hull))
24
25 points = [(1,1),(4,6),(8,1),(0,0),(3,3)]
26 print(convex_hull(points))
```

Output

```
[(0, 0), (4, 6), (8, 1)]
=== Code Execution Successful ===
```

Result

Thus, the convex hull of the given set of points was successfully determined using the brute-force approach.

Time Complexity: $O(n^3)$

Space Complexity: $O(n)$

Experiment 12 – Travelling Salesman Problem Using Exhaustive Search

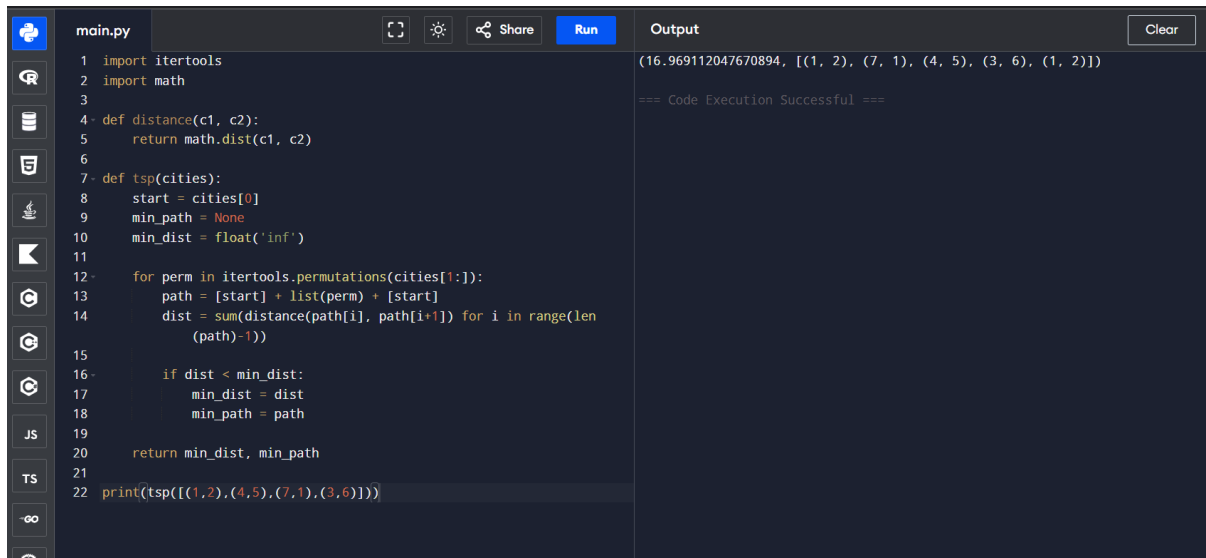
Aim

To solve the Travelling Salesman Problem (TSP) using exhaustive search and find the shortest possible route that visits every city exactly once and returns to the starting city.

Algorithm

1. Start.
2. Define a function to calculate the distance between two cities.
3. Select the first city as the starting city.
4. Generate all possible permutations of the remaining cities.
5. Construct a complete route for each permutation.
6. Calculate the total distance of each route.
7. Compare each route with the current shortest route.
8. Store the route with minimum distance.
9. Return to the starting city.
10. Display the shortest path and distance.
11. Stop.

Output



```
main.py
1 import itertools
2 import math
3
4 def distance(c1, c2):
5     return math.dist(c1, c2)
6
7 def tsp(cities):
8     start = cities[0]
9     min_path = None
10    min_dist = float('inf')
11
12    for perm in itertools.permutations(cities[1:]):
13        path = [start] + list(perm) + [start]
14        dist = sum(distance(path[i], path[i+1]) for i in range(len(path)-1))
15
16        if dist < min_dist:
17            min_dist = dist
18            min_path = path
19
20    return min_dist, min_path
21
22 print(tsp([(1,2),(4,5),(7,1),(3,6)]))
```

Output

```
(16.969112047670894, [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)])
```

=== Code Execution Successful ===

Result

Thus, the TSP was successfully solved using exhaustive search by examining all possible routes and selecting the route with minimum total distance.

Time Complexity

$O(n!)$ route permutations, with $O(n)$ work to evaluate each route, giving approximately $O(n \times n!)$.

Space Complexity

$O(n)$ excluding the permutations generated by the library; including permutation storage/generation, it can be $O(n \times n!)$ depending on implementation.

Experiment 13 – Assignment Problem Using Exhaustive Search

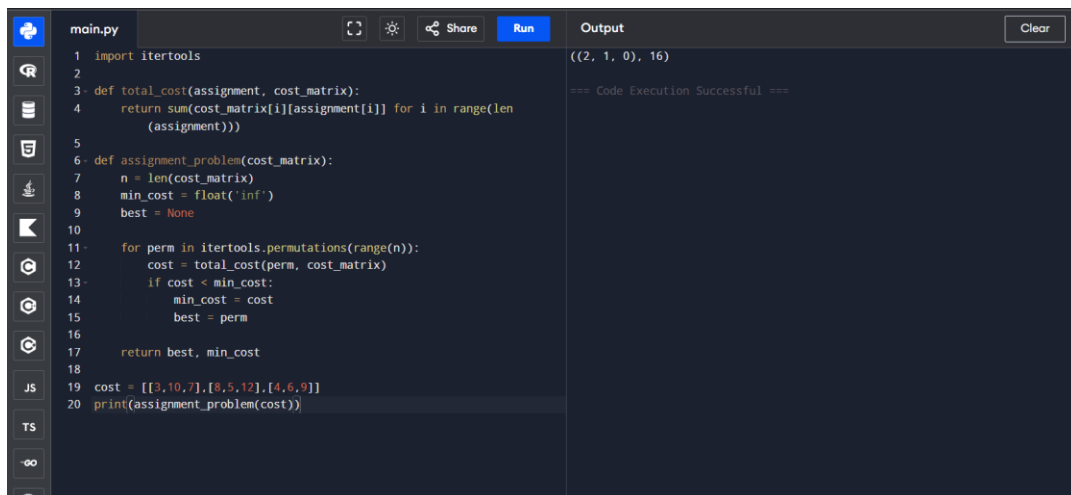
Aim

To solve the Assignment Problem using exhaustive search by assigning each worker to a unique task while minimizing the total assignment cost.

Algorithm

1. Start.
2. Read the cost matrix.
3. Generate all possible task assignments using permutations.
4. Calculate the total cost for each assignment.
5. Compare each cost with the current minimum cost.
6. Store the assignment having the minimum cost.
7. Display the optimal assignment and total cost.
8. Stop.

Output



```
main.py
1 import itertools
2
3 def total_cost(assignment, cost_matrix):
4     return sum(cost_matrix[i][assignment[i]] for i in range(len(
5         assignment)))
6
7 def assignment_problem(cost_matrix):
8     n = len(cost_matrix)
9     min_cost = float('inf')
10    best = None
11
12    for perm in itertools.permutations(range(n)):
13        cost = total_cost(perm, cost_matrix)
14        if cost < min_cost:
15            min_cost = cost
16            best = perm
17
18    return best, min_cost
19
20 cost = [[3,10,7],[8,5,12],[4,6,9]]
21 print(assignment_problem(cost))
```

Output

```
((2, 1, 0), 16)
```

=== Code Execution Successful ===

Result

Thus, the optimal worker-task assignment with minimum total cost was successfully obtained using exhaustive search.

Time Complexity

$O(n \times n!)$

Space Complexity

$O(n)$ auxiliary space, excluding generated permutations.

Experiment 14 – 0/1 Knapsack Using Exhaustive Search

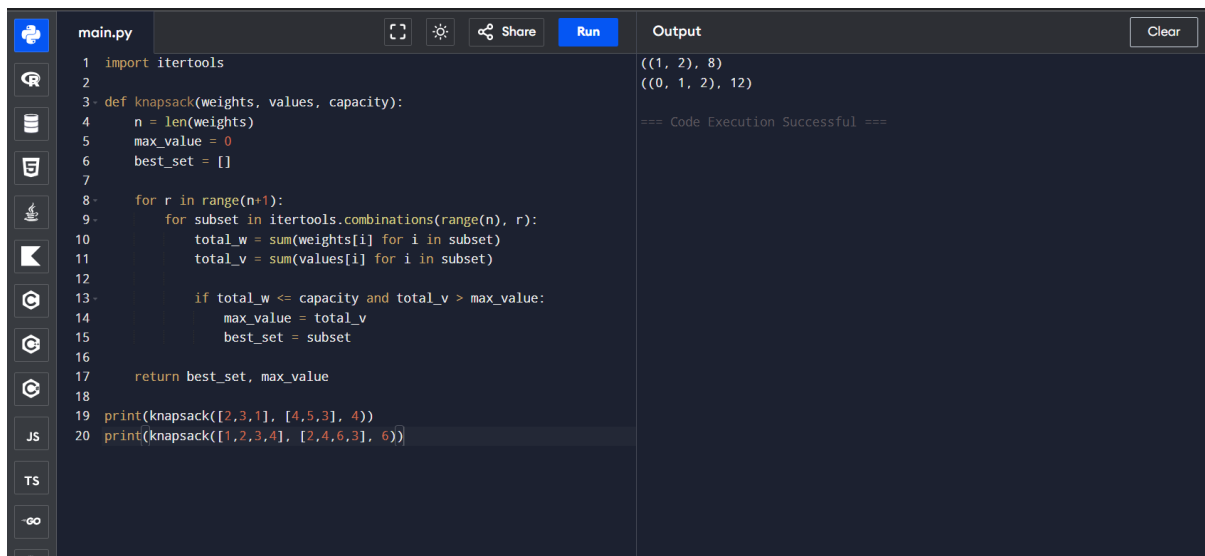
Aim

To solve the 0/1 Knapsack Problem using exhaustive search by finding the subset of items with maximum value without exceeding the given capacity.

Algorithm

1. Start.
2. Read the weights, values, and knapsack capacity.
3. Generate all possible subsets of items.
4. Calculate the total weight of each subset.
5. Check whether the subset is within the given capacity.
6. Calculate the total value of every feasible subset.
7. Compare the values and retain the subset with maximum value.
8. Display the optimal selection and total value.
9. Stop.

Output



```
main.py
1 import itertools
2
3 def knapsack(weights, values, capacity):
4     n = len(weights)
5     max_value = 0
6     best_set = []
7
8     for r in range(n+1):
9         for subset in itertools.combinations(range(n), r):
10             total_w = sum(weights[i] for i in subset)
11             total_v = sum(values[i] for i in subset)
12
13             if total_w <= capacity and total_v > max_value:
14                 max_value = total_v
15                 best_set = subset
16
17     return best_set, max_value
18
19 print(knapsack([2,3,1], [4,5,3], 4))
20 print(knapsack([1,2,3,4], [2,4,6,3], 6))
```

Output

```
((1, 2), 8)
((0, 1, 2), 12)

=== Code Execution Successful ===
```

Result

Thus, the 0/1 Knapsack Problem was successfully solved using exhaustive search, and the subset having the maximum value within the capacity was obtained.

Time Complexity: $O(2^n \times n)$

Space Complexity: $O(n)$ auxiliary space.